

Reviewer Pack — Minimal Rank of Generalized Polynomials vs Boolean Circuit W...

Entry be3fb0e7cddc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 04:07:01 UTC

Minimal Rank of Generalized Polynomials vs Boolean Circuit Weights

Entry ID: be3fb0e7cddc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 04:07:01 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Generalized Polynomials) **Field B** (complexity object): Complexity Theory: Boolean Circuit Complexity

Statement:

{‘s1’: ‘For every generalized polynomial f over the Boolean ring with degree D , let $\rho(f)$ be its rank in the Grothendieck-Witt ring. For all $n \leq 40$, there exists a boolean circuit of depth D with weight at most $2^{\rho(f)}$.’, ‘s2’: ‘Additionally, if two generalized polynomials f and g have the same degree and their ranks $\rho(f) = \rho(g)$, then any boolean circuit computing f must have the same weight as any boolean circuit computing g .’, ‘s3’: ‘This holds true for all generalized polynomials that can be expressed in terms of symmetric functions or Schur polynomials.’}

Rationale (proposer’s reasoning):

{‘s1’: ‘Generalized polynomials provide a rich algebraic structure that has not been extensively explored in the context of circuit complexity.’, ‘s2’: ‘The Grothendieck-Witt ring is an invariant that captures essential information about the geometric properties of generalized polynomials, which may reveal insights into their computational complexity.’, ‘s3’: ‘By linking these algebraic concepts with boolean circuits, we aim to uncover new relationships between algebraic structure and circuit complexity.’}

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): adbc109431422c0a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a generalized polynomial f , if the difference between its rank $\rho(f)$ in the Grothendieck-Witt ring and the maximum weight of circuits computing it is at most 3, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal rank generalized polynomials AND Boolean circuit weights - Grothendieck-Witt ring rank AND boolean circuit depth - symmetric functions Schur polynomials AND circuit weight comparison

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(m):
                if i != j:
                    factor = A[j][i] / A[i][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def rank(A):
        A = gaussian_elimination(A)
        return sum(1 for row in A if any(row))

    def boolean_circuit_weight(f, n):
        # Simplified model of a boolean circuit weight
        # This is a placeholder and should be replaced with actual circuit construction logic
        return 2 ** rank(f)

    def generate_polynomial(n):
        # Generate a random polynomial over the Boolean ring
        # This is a simplified model and should be replaced with actual polynomial generation logic
```

```

        return [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

n = random.randint(5, 40)
f = generate_polynomial(n)
rho_f = rank(f)
circuit_weight = boolean_circuit_weight(f, n)

if abs(rho_f - circuit_weight) > 3:
    return {
        "metric_name": "Rank vs Circuit Weight",
        "metric_value": rho_f,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"rho(f)={rho_f}, circuit_weight={circuit_weight}"
    }
else:
    return {
        "metric_name": "Rank vs Circuit Weight",
        "metric_value": rho_f,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        results.append(result)
        print(f"TRIAL: {result}")

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a559851b.py", line 75, in <module>

```

    result = run_trial(seed)
              ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a559851b.py", line 49, in run_trial
    rho_f = rank(f)
              ^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a559851b.py", line 34, in rank
    A = gaussian_elimination(A)
          ^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a559851b.py", line 28, in gaussian_elim
    factor = A[j][i] / A[i][i]
              ^~~~~~^~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error before producing data, which means it did not complete the computation necessary to evaluate the conjecture. | next: Investigate and fix the cause of the division by zero error in the code. Once fixed, rerun the test to verify if it supports or falsifies the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15128
2	propose	ollama_remote	glm4:latest	0	0	14354
3	preregistration	ollama_remote	glm4:latest	0	0	8998
4	novelty	ollama_remote	glm4:latest	0	0	8225
5	novelty	ollama_remote	glm4:latest	0	0	9487
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19887
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10862
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10080
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9242
10	judge	ollama_remote	glm4:latest	0	0	15851

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122115 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/be3fb0e7cddc.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/be3fb0e7cddc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/be3fb0e7cddc.tar.gz` (if generated)